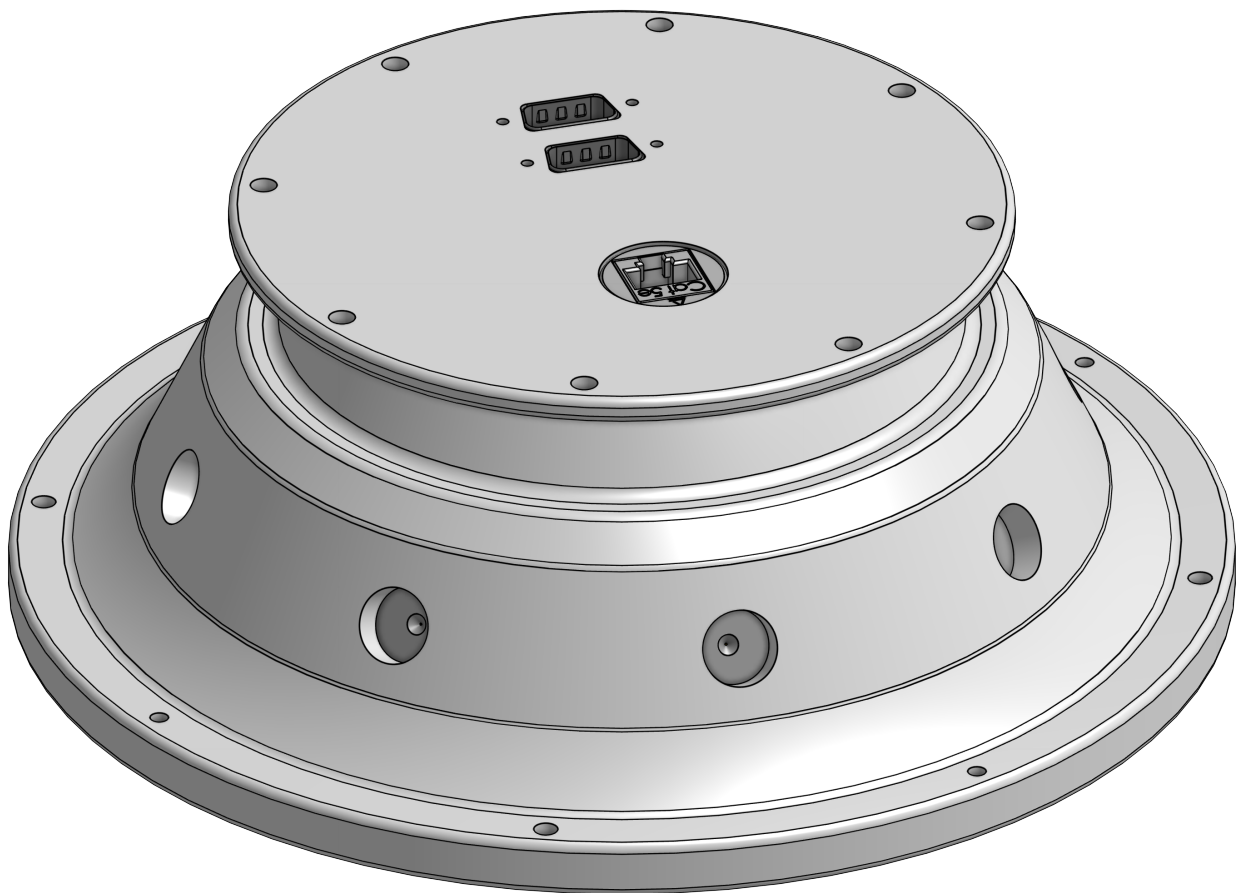


PINHOLE SUN SENSOR ARRAY MANUAL



June 2024



1 Preface

**Thank you for working with StarSpec Technologies!
We appreciate your partnership!**

This manual outlines the functionality and purpose of the Pinhole Sun Sensor Array (PSS Array).

The PSS Array is developed for Louisiana State University for use on the Antarctic Demonstrator for the Advanced Particle Physics Telescope (ADAPT) Long Duration Balloon (LDB) project.

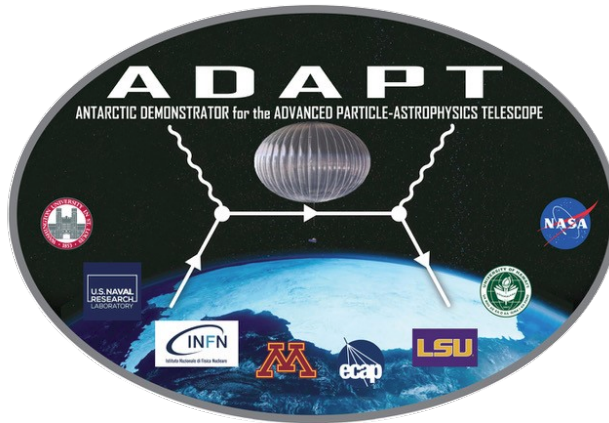


Figure 1: ADAPT Mission Patch

StarSpec has strong heritage and demonstrated success in high fidelity pointing stability, control and reconstruction for balloon-borne instrumentation at high- and mid-latitudes (LDB and SPB).

This includes full-sky pointing reconstruction expertise for large, highly constrained astrophysics balloon-borne payloads, including SuperBIT, EXCITE, Spider 1/2, BLAST, BOOMERanG, etc. Also, StarSpec has degree, arcminute, arcsecond, and sub-arcsecond reconstruction, feedback, and control capabilities.



Figure 2: Spider 2—Antarctica 2022/23 LDB Campaign



Figure 3: SuperBIT—Wanaka 2023 SPB Campaign

Read this manual before use

This manual can provide answers and guidance for use of the PSS Array. Other questions can be directed to StarSpec Technologies.

1.1 Handling & Storage

The PSS Array is self housed and mechanically self contained for balloon-borne operations. Thus, there are specific handling and storage considerations that must be followed.

1.1.1 Pinhole Care

Caution should be taken when handling the PSS array as to not damage any of the 6 pinhole sub-assemblies. This concerns the pinhole geometry primarily, the user must ensure that the assembly is stored and operated in an environment that will optimize performance. This includes:

- Storage and Operation in a Humidity free Environment.
- Caution to possible debris entering pinhole.

1.1.2 Housing Care

The housing encompasses all the Pinhole sub-assemblies and the electronics stack. For nominal operations, it is crucial that the housing configuration is maintained. Thus:

- Do not tighten / remove any screws or plugs.
- Take care that no objects are inserted into housing apertures.
- Aperture stickers are only to be removed before flight.

1.1.3 General Cautions

- Do not use the product in a wet or humid environment.
- Do not attempt to disassemble or alter the equipment unless advised by StarSpec Technologies.
- Ensure rigid mounting to interfaces using the four mounting holes.



CONTENTS

1	Preface	i
1.1	Handling & Storage	ii
1.1.1	Pinhole Care	ii
1.1.2	Housing Care	ii
1.1.3	General Cautions	ii
	List of Acronyms	1
	List of Figures	2
	List of Tables	2
2	Theory of Operation	3
3	High Level Mechanical	4
4	Performance	5
4.1	Specifications	5
5	Components	5
5.1	GPS Input-Required	5
5.2	Tilt Sensor	6
6	Pinouts & Settings	6
7	Commanding & Telemetry Data Formats	7
7.1	Commands	7
7.1.1	Command Details	7
7.1.2	Command Descriptions	7
7.2	Telemetry	12
7.2.1	Telemetry Details	12
7.2.2	Telemetry Descriptions	12
8	Sample Programs & API	16
8.1	Connecting for the First Time	16
8.2	Parsing the Byte Stream	17
8.3	Sending Commands	21
9	Calibration	23
9.1	Stage 1 Calibration - Individual PSS Linearization	23
9.2	Stage 2 Calibration - Az/EI Correlation	23

List of Acronyms

ADC	Analog-Digital Converter
ADAPT	Antarctic Demonstrator for the Advanced Particle-Astrophysics Telescope
ASCII	American Standard Code for Information Interchange
Az	Azimuth
BLAST	Balloon-borne Large APerture Submillimeter Telescope
BOOMERanG	Balloon Observations Of Millimetric Extragalactic Radiation and Geophysics
EI	Elevation
EXCITE	Exoplanet Climate Infrared Telescope
FDC	Full-Duplex Communication
FSW	Flight Software
GGA	Global Positioning System Fix Data (latitude, longitude, altitude)
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
LDB	Long Duration Balloon
MCC	Mission Control Center
NMEA	National Marine Electronics Association
PSD	Position Sensitive Device
PSS	Pinhole Sun Sensor
RMC	Recommended Minimum Specific GNSS Data (year, month, day, hours, minutes, seconds)
RPi	Raspberry Pi
SCC	Single Chip Computer
SPB	Super Pressure Balloon
SuperBIT	Super-pressure Balloon-borne Imaging Telescope
UDP	User Datagram Protocol

List of Figures

1	ADAPT Mission Patch	i
2	Spider 2—Antarctica 2022/23 LDB Campaign	i
3	SuperBIT—Wanaka 2023 SPB Campaign	i
4	Azimuth & Elevation in the Balloon Gondala Context	3
5	Top Level Assembly of PSS Array	4
6	SCL3300-D01 Tilt Sensor	6
7	Serial Connection	6
8	Power Connection	7
9	PSS 41 Heat Map	23

List of Tables

1	Performance Specifications for PSS Array	5
2	Operational & Interface Specifications	5
3	Serial Connection Pinout	6
4	Power Connection Pinout	7
5	Command Data Types	7
6	Configure ADC 0 filtering	8
7	Configure ADC 1 filtering	8
8	Configure ADC 2 filtering	8
9	Configure ADC 3 filtering	8
10	Configure ADC 4 filtering	8
11	Configure ADC 5 filtering	8
12	gps: Connect the GPS	9
13	Set the PSS 0 Threshold parameters for AzEl data	9
14	Set the PSS 1 Threshold parameters for AzEl data	9
15	Set the PSS 2 Threshold parameters for AzEl data	9
16	Set the PSS 3 Threshold parameters for AzEl data	9
17	Set the PSS 4 Threshold parameters for AzEl data	10
18	Set the PSS 5 Threshold parameters for AzEl data	10
19	Set the command IP:port for the PSS array.	10
20	Set the Az offset by asserting the current Az of the PSS array	10
21	Set the Az offset for the PSS array.	10
22	Set the telemetry IP:port for the PSS array.	10
23	Command the calibration table to a given angle (only available when calibration table is connected)	11
24	Command the calibration table to stop motion and active control, yielding manual motion (only available when calibration table is connected)	11
25	Command the calibration table to set the current position to the desired value (only available when calibration table is connected)	11
26	Command the calibration table to cycle a trapezoidal position profile from start to end for a given number of cycles (only available when calibration table is connected)	11
27	Command the calibration table to have a given maximum speed for all moves - this command does not issue a move, but this speed is used for all goto or cycle commands (only available when calibration table is connected)	11
28	Miscellaneous / debugging commands (no parameters)	12
29	Telemetry Data Types	12
30	Telemetry Items – total frame size of 833 Bytes	16

2 Theory of Operation

The PSS Array provides a 240 degree azimuth (Az) Sun position tracker with onboard computational resources for operating stand-alone software. Additionally, the device presents elevation position determination for a local Sun elevation angle of 25 ± 10 degrees, consistent with Antarctic LDB flight in the austral summer.

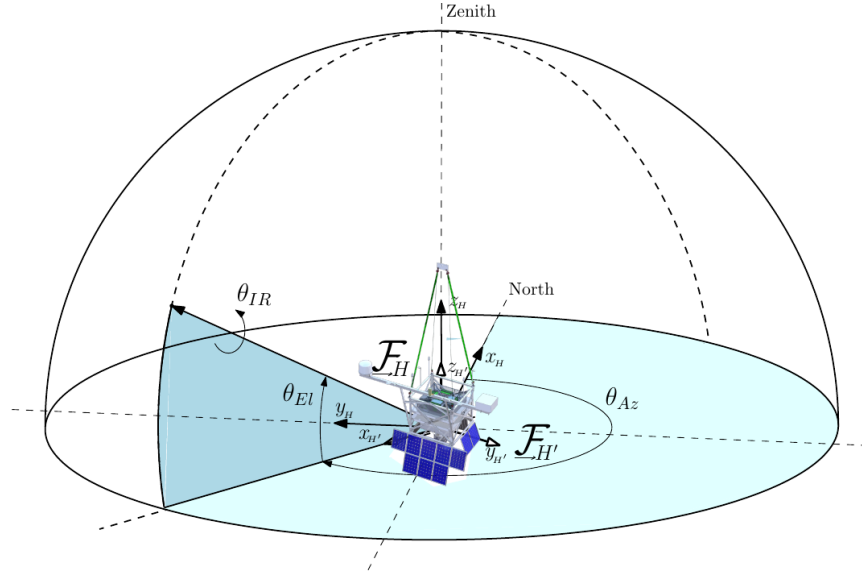


Figure 4: Azimuth & Elevation in the Balloon Gondala Context

The independent flight software records and determines time stamped Az/EI coordinates. For determining these coordinates, inputs are required by the FSW: RS232 GPS, tip and tilt angles from the horizontal, and time of day. The tip and tilt angles are read from an internal tilt sensor on the custom daughter board mounted to the Raspberry Pi. The PSS uses modularized packaging of the above elements for robust operations in the stratosphere.

The FSW comprises of a commanding interface for controlling, calibrating, tuning, and regular operations of the PSS.

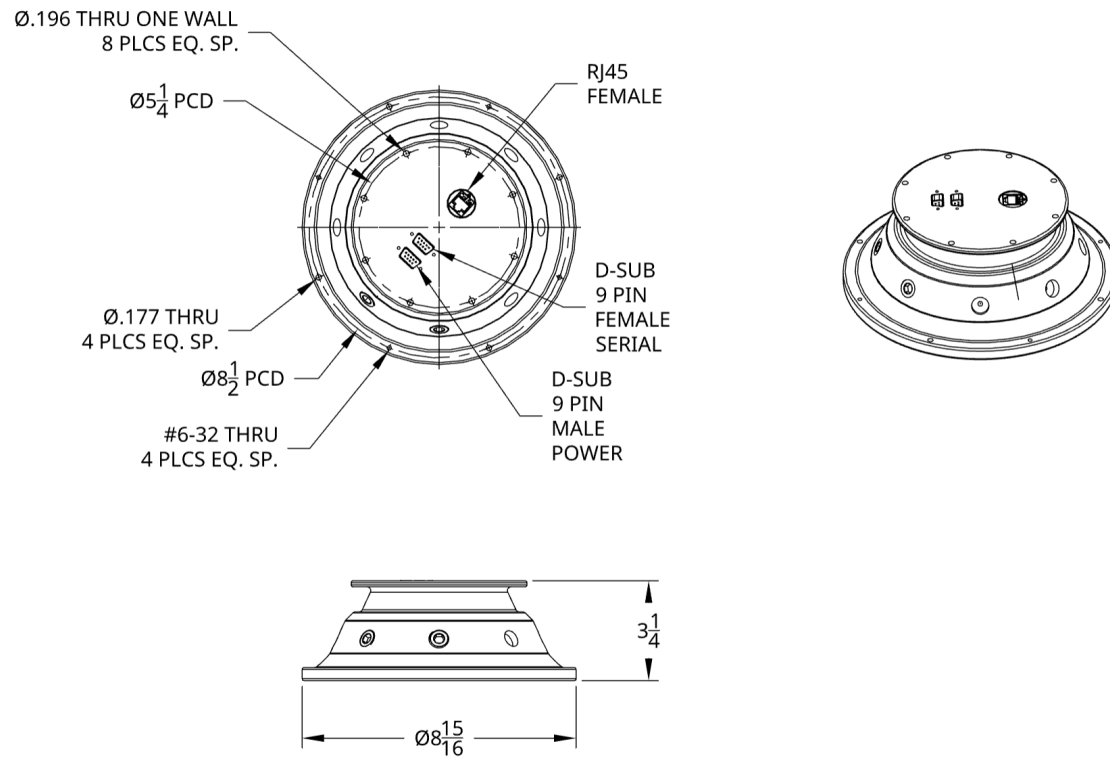
The design incorporates pinhole analog sensors, a split array mechanical design, Ethernet interface, and data synchronization via payload.

The Pinhole Sun Sensor provides the Az/EI Sun orientation information relative to the sensor orientation. The Sun is imaged through a series of pinholes onto 2-dimensional position sensitive detectors (PSD) whose outputs are used to calculate the x and y position of the image. The x and y positions are then used to get information about the module's relative orientation.

The PSS Array takes the GPS readings along with the x and y coordinates, and translates them to the Array's orientation information: Azimuth and Elevation.

3 High Level Mechanical

Top level assembly of the PSS Array can be seen below.



UNLESS OTHERWISE SPECIFIED, DIMENSIONS ARE IN INCHES. CONTACT STARSPEC FOR TOLERANCES.

ADAPT.031.ASM.DRW.005	REV	A	SCALE	NTS	SHEET	3 of 3	REV	D
-----------------------	-----	---	-------	-----	-------	--------	-----	---

Figure 5: Top Level Assembly of PSS Array

4 Performance

4.1 Specifications

Table 1: Performance Specifications for PSS Array

Performance Specifications	Min.	Nom.	Max.	Units
Nominal Angular Resolution	1.10	5.00	–	arcminute
Relative Solar Azimuth Range	–	240	270	degrees
Solar Elevation Range	15	25	35	degrees
Az / El Determination Rate	–	5	20	Hz
Leveling precision	0.05	0.1	–	degree
Leveling offset	0.0	0.3	5.0	degree

Table 2: Operational & Interface Specifications

Operational Specifications	Min.	Nom.	Max.	Units
Supply voltage range	18	48	70	Volt
Peak Power	–	3.6	7.2	Watt
Temperature Range	0	25	85	Celsius
Required Input GPS Precision	–	–	5	arcminute
Required Input GPS Rate	1	–	–	Hz
Required Mounting Precision	–	–	0.1	degrees

5 Components

5.1 GPS Input-Required

A GPS with the following parameters is required:

Expected Baud Rate : 9600 bps

Expected Message Format: GGA and RMC NMEA sentences.

Expected Communication Interface: RS232

Required standard NMEA sentences (1 Hz recommended):

GGA: Global Positioning System Fix Data (latitude, longitude, altitude)

RMC: GNSS Timing Data (year, month, day, hours, minutes, seconds)

A resource for the NMEA standard can be found online (https://receiverhelp.trimble.com/alloy-gnss/en-us/NMEA-0183messages_MessageOverview.html) or is available upon request.

5.2 Tilt Sensor

The tilt sensor is a SCL3300-D01 3-axis inclinometer which measures tip and tilt angles so that the FSW can determine the position of the PSS with respect to the sun. Inclination ranges are limited in Mode 4 to maximum $\pm 10^\circ$.



Figure 6: SCL3300-D01 Tilt Sensor

More details on the SCL3300-D01 can be found online (<https://www.digikey.ca/en/products/detail/murata-electronics/SCL3300-D01-1/9950616>) or available upon request.

6 Pinouts & Settings

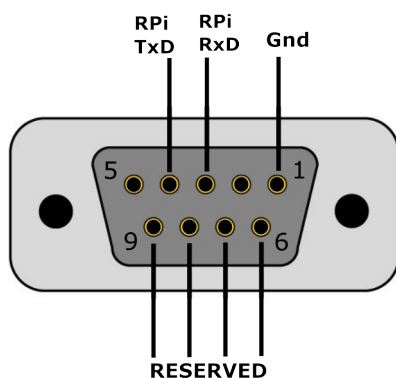


Figure 7: Serial Connection

GPS RS232 Signal Input DB9-Female	J2 Pin Number
Ground	Pin 1
N.C	Pin 2
RxD	Pin 3
TxD	Pin 4
N.C.	Pin 5
RESERVED	Pin 6
RESERVED	Pin 7
RESERVED	Pin 8
RESERVED	Pin 9

Table 3: Serial Connection Pinout

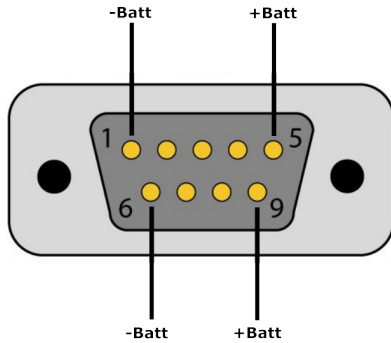


Figure 8: Power Connection

Battery/Gondola Power Input DB9-Male	J3 Pin Number
-Batt	Pin 1
N.C.	Pin 2
N.C.	Pin 3
N.C.	Pin 4
+Batt	Pin 5
-Batt	Pin 6
N.C.	Pin 7
N.C.	Pin 8
+Batt	Pin 9
Additional Ports	
Ethernet	

Table 4: Power Connection Pinout

7 Commanding & Telemetry Data Formats

Commands are transmitted to the PSS array as UDP packets over port X within the flight network.

7.1 Commands

Commands for the PSS are in a simple ASCII format. Numerical parameters are space-separated following the ASCII command (e.g. "adc_0_filter 20" to set the ADC filter boxcar size to 20). String or character parameters are also space-separated, but must be quoted (e.g. "gps_connect 115200 "/dev/ttyS0"" to set the GPS baud rate to 115.2 kbaud and the device path to /dev/ttyS0).

7.1.1 Command Details

Type	Description
i	16 bit signed integer
l	32 bit unnormalized integer
f	16 bit normalized floating point
d	32 bit normalized floating point
s	7-bit character string

Table 5: Command Data Types

7.1.2 Command Descriptions

Commands are specified in tables below.

Command: adc_0.filter				
Parameter	Type	(Min,Max)	Default	Description
boxcar_size_adc_0	i	(1,1000)	0	Boxcar size:

Table 6: Configure ADC 0 filtering

Command: adc_1.filter				
Parameter	Type	(Min,Max)	Default	Description
boxcar_size_adc_1	i	(1,1000)	0	Boxcar size:

Table 7: Configure ADC 1 filtering

Command: adc_2.filter				
Parameter	Type	(Min,Max)	Default	Description
boxcar_size_adc_2	i	(1,1000)	0	Boxcar size:

Table 8: Configure ADC 2 filtering

Command: adc_3.filter				
Parameter	Type	(Min,Max)	Default	Description
boxcar_size_adc_3	i	(1,1000)	0	Boxcar size:

Table 9: Configure ADC 3 filtering

Command: adc_4.filter				
Parameter	Type	(Min,Max)	Default	Description
boxcar_size_adc_4	i	(1,1000)	0	Boxcar size:

Table 10: Configure ADC 4 filtering

Command: adc_5.filter				
Parameter	Type	(Min,Max)	Default	Description
boxcar_size_adc_5	i	(1,1000)	0	Boxcar size:

Table 11: Configure ADC 5 filtering

Command: gps_connect				
Parameter	Type	(Min,Max)	Default	Description
baud_gps	I	(0,115200)	115200	gps: Baud rate (bps) for GPS

Table 12: *gps: Connect the GPS*

Command: pss_0.threshold				
Parameter	Type	(Min,Max)	Default	Description
thresh_pss_0	d	(0,10)	0.4	pss: Per channel threshold [V]
radius_pss_0	d	(0,20)	2.15	pss: Per PSS sensor radius threshold [mm]

Table 13: *Set the PSS 0 Threshold parameters for AzEl data*

Command: pss_1.threshold				
Parameter	Type	(Min,Max)	Default	Description
thresh_pss_1	d	(0,10)	0.4	pss: Per channel threshold [V]
radius_pss_1	d	(0,20)	2.15	pss: Per PSS sensor radius threshold [mm]

Table 14: *Set the PSS 1 Threshold parameters for AzEl data*

Command: pss_2.threshold				
Parameter	Type	(Min,Max)	Default	Description
thresh_pss_2	d	(0,10)	0.4	pss: Per channel threshold [V]
radius_pss_2	d	(0,20)	2.15	pss: Per PSS sensor radius threshold [mm]

Table 15: *Set the PSS 2 Threshold parameters for AzEl data*

Command: pss_3.threshold				
Parameter	Type	(Min,Max)	Default	Description
thresh_pss_3	d	(0,10)	0.4	pss: Per channel threshold [V]
radius_pss_3	d	(0,20)	2.15	pss: Per PSS sensor radius threshold [mm]

Table 16: *Set the PSS 3 Threshold parameters for AzEl data*

Command: pss_4_threshold				
Parameter	Type	(Min,Max)	Default	Description
thresh_pss_4	d	(0,10)	0.4	pss: Per channel threshold [V]
radius_pss_4	d	(0,20)	2.15	pss: Per PSS sensor radius threshold [mm]

Table 17: Set the PSS 4 Threshold parameters for AzEl data

Command: pss_5_threshold				
Parameter	Type	(Min,Max)	Default	Description
thresh_pss_5	d	(0,10)	0.4	pss: Per channel threshold [V]
radius_pss_5	d	(0,20)	2.15	pss: Per PSS sensor radius threshold [mm]

Table 18: Set the PSS 5 Threshold parameters for AzEl data

Command: set_cmd_address				
Parameter	Type	(Min,Max)	Default	Description
override_cmd	s	("0.0.0.0:12000", "255.255.255.255:60000")	10.0.0.2:54345	Set the IP:port on which to receive commands

Table 19: Set the command IP:port for the PSS array.

Command: set_current_az				
Parameter	Type	(Min,Max)	Default	Description
current_az	d	(-360,360)	0	Current Az (deg):

Table 20: Set the Az offset by asserting the current Az of the PSS array

Command: set_offset_az				
Parameter	Type	(Min,Max)	Default	Description
offset_az	d	(-360,360)	0	Az offset (deg):

Table 21: Set the Az offset for the PSS array.

Command: set_tlm_address				
Parameter	Type	(Min,Max)	Default	Description
override_tlm	s	("0.0.0.0:12000", "255.255.255.255:60000")	10.0.0.1:53335	Set the IP:port on which to receive telemetry data

Table 22: Set the telemetry IP:port for the PSS array.

Command: table_goto				
Parameter	Type	(Min,Max)	Default	Description
cmd_position_table	d	(-360,360)	0	Commanded table position (deg):

Table 23: Command the calibration table to a given angle (only available when calibration table is connected)

Command: table_stop				
Parameter	Type	(Min,Max)	Default	Description
—	—	—	—	—

Table 24: Command the calibration table to stop motion and active control, yielding manual motion (only available when calibration table is connected)

Command: table_set				
Parameter	Type	(Min,Max)	Default	Description
set_position_table	d	(-360,360)	0	Current table position (deg):

Table 25: Command the calibration table to set the current position to the desired value (only available when calibration table is connected)

Command: table_cycle				
Parameter	Type	(Min,Max)	Default	Description
cycle_start_table	d	(-360,360)	0	Starting position for table cycle (deg):
cycle_end_table	d	(-360,360)	0	Ending position for table cycle (deg):
cycles_left_table	d	(-360,360)	0	Number of cycles to execute (negative for infinite):

Table 26: Command the calibration table to cycle a trapezoidal position profile from start to end for a given number of cycles (only available when calibration table is connected)

Command: table_set_speed				
Parameter	Type	(Min,Max)	Default	Description
cmd_speed_table	d	(0,90)	0	Table speed (deg):

Table 27: Command the calibration table to have a given maximum speed for all moves - this command does not issue a move, but this speed is used for all goto or cycle commands (only available when calibration table is connected)

Command	Description
adc_0_force_reset	Force a reset of the ADC
adc_1_force_reset	Force a reset of the ADC
adc_2_force_reset	Force a reset of the ADC
adc_3_force_reset	Force a reset of the ADC
adc_4_force_reset	Force a reset of the ADC
adc_5_force_reset	Force a reset of the ADC
gps_disconnect	Disconnect the GPS
reset_factory_defaults	Reset all the settings to factory defaults
restart	Restart the main PSS program

Table 28: Miscellaneous / debugging commands (no parameters)

7.2 Telemetry

The data type used for Telemetry items is Binary. The telemetry packets are continuously transmitted over port 54345 via UDP to the FDC. The contents of the packets themselves are outlined in Section 7.2.2, alongside the allocated bytes, types, and names.

7.2.1 Telemetry Details

Below is a type list for the following sections describing telemetry. Min and max values are specified for some floating point types for informational purposes only.

Type	Description
s	16 bit signed integer
u	16 bit unsigned integer
c	8 bit character / unsigned integer
i	32 bit signed integer
d	64 bit floating point
f	32 bit float

Table 29: Telemetry Data Types

7.2.2 Telemetry Descriptions

In the table below the contents of the telemetry packet is outlined. Each entry has its own name, type, minimum and maximum value (if applicable), samples-per-frame (spf), size, description, and units. It is notable that the byte transmission uses little endian system.

Note: All *_table items are only valid when connected to a calibration table and are empty otherwise.

Name	Type	(min/max)	Spf	Start Byte	Bytes	Description	Units
v_x1_pss_0	d	(0.0, 50.0)	1	0	8	Voltage	V
v_x2_pss_0	d	(0.0, 50.0)	1	8	8	Voltage	V
v_y1_pss_0	d	(0.0, 50.0)	1	16	8	Voltage	V
v_y2_pss_0	d	(0.0, 50.0)	1	24	8	Voltage	V
v_sum_pss_0	d	(0.0, 50.0)	1	32	8	Voltage	V
thresh_pss_0	d	(0.0, 10.0)	1	40	8	–	–
radius_pss_0	d	(0.0, 20.0)	1	48	8	–	–
good_pss_0	c	–	1	56	1	–	–
x_raw_pss_0	d	–	1	57	8	–	–
y_raw_pss_0	d	–	1	65	8	–	–
x_corr_pss_0	d	–	1	73	8	–	–
y_corr_pss_0	d	–	1	81	8	–	–
az_pss_0	d	(0.0, 360.0)	1	89	8	Angle	deg
el_pss_0	d	(0.0, 360.0)	1	97	8	Angle	deg
valid_pss_0	c	–	1	105	1	–	–
v_x1_pss_1	d	(0.0, 50.0)	1	106	8	Voltage	V
v_x2_pss_1	d	(0.0, 50.0)	1	114	8	Voltage	V
v_y1_pss_1	d	(0.0, 50.0)	1	122	8	Voltage	V
v_y2_pss_1	d	(0.0, 50.0)	1	130	8	Voltage	V
v_sum_pss_1	d	(0.0, 50.0)	1	138	8	Voltage	V
thresh_pss_1	d	(0.0, 10.0)	1	146	8	–	–
radius_pss_1	d	(0.0, 20.0)	1	154	8	–	–
good_pss_1	c	–	1	162	1	–	–
x_raw_pss_1	d	–	1	163	8	–	–
y_raw_pss_1	d	–	1	171	8	–	–
x_corr_pss_1	d	–	1	179	8	–	–
y_corr_pss_1	d	–	1	187	8	–	–
az_pss_1	d	(0.0, 360.0)	1	195	8	Angle	deg
el_pss_1	d	(0.0, 360.0)	1	203	8	Angle	deg
valid_pss_1	c	–	1	211	1	–	–
v_x1_pss_2	d	(0.0, 50.0)	1	212	8	Voltage	V
v_x2_pss_2	d	(0.0, 50.0)	1	220	8	Voltage	V

v_y1_pss_2	d	(0.0, 50.0)	1	228	8	Voltage	V
v_y2_pss_2	d	(0.0, 50.0)	1	236	8	Voltage	V
v_sum_pss_2	d	(0.0, 50.0)	1	244	8	Voltage	V
thresh_pss_2	d	(0.0, 10.0)	1	252	8	–	–
radius_pss_2	d	(0.0, 20.0)	1	260	8	–	–
good_pss_2	c	–	1	268	1	–	–
x_raw_pss_2	d	–	1	269	8	–	–
y_raw_pss_2	d	–	1	277	8	–	–
x_corr_pss_2	d	–	1	285	8	–	–
y_corr_pss_2	d	–	1	293	8	–	–
az_pss_2	d	(0.0, 360.0)	1	301	8	Angle	deg
el_pss_2	d	(0.0, 360.0)	1	309	8	Angle	deg
valid_pss_2	c	–	1	317	1	–	–
v_x1_pss_3	d	(0.0, 50.0)	1	318	8	Voltage	V
v_x2_pss_3	d	(0.0, 50.0)	1	326	8	Voltage	V
v_y1_pss_3	d	(0.0, 50.0)	1	334	8	Voltage	V
v_y2_pss_3	d	(0.0, 50.0)	1	342	8	Voltage	V
v_sum_pss_3	d	(0.0, 50.0)	1	350	8	Voltage	V
thresh_pss_3	d	(0.0, 10.0)	1	358	8	–	–
radius_pss_3	d	(0.0, 20.0)	1	366	8	–	–
good_pss_3	c	–	1	374	1	–	–
x_raw_pss_3	d	–	1	375	8	–	–
y_raw_pss_3	d	–	1	383	8	–	–
x_corr_pss_3	d	–	1	391	8	–	–
y_corr_pss_3	d	–	1	399	8	–	–
az_pss_3	d	(0.0, 360.0)	1	407	8	Angle	deg
el_pss_3	d	(0.0, 360.0)	1	415	8	Angle	deg
valid_pss_3	c	–	1	423	1	–	–
v_x1_pss_4	d	(0.0, 50.0)	1	424	8	Voltage	V
v_x2_pss_4	d	(0.0, 50.0)	1	432	8	Voltage	V
v_y1_pss_4	d	(0.0, 50.0)	1	440	8	Voltage	V
v_y2_pss_4	d	(0.0, 50.0)	1	448	8	Voltage	V

v_sum_pss_4	d	(0.0, 50.0)	1	456	8	Voltage	V
thresh_pss_4	d	(0.0, 10.0)	1	464	8	–	–
radius_pss_4	d	(0.0, 20.0)	1	472	8	–	–
good_pss_4	c	–	1	480	1	–	–
x_raw_pss_4	d	–	1	481	8	–	–
y_raw_pss_4	d	–	1	489	8	–	–
x_corr_pss_4	d	–	1	497	8	–	–
y_corr_pss_4	d	–	1	505	8	–	–
az_pss_4	d	(0.0, 360.0)	1	513	8	Angle	deg
el_pss_4	d	(0.0, 360.0)	1	521	8	Angle	deg
valid_pss_4	c	–	1	529	1	–	–
v_x1_pss_5	d	(0.0, 50.0)	1	530	8	Voltage	V
v_x2_pss_5	d	(0.0, 50.0)	1	538	8	Voltage	V
v_y1_pss_5	d	(0.0, 50.0)	1	546	8	Voltage	V
v_y2_pss_5	d	(0.0, 50.0)	1	554	8	Voltage	V
v_sum_pss_5	d	(0.0, 50.0)	1	562	8	Voltage	V
thresh_pss_5	d	(0.0, 10.0)	1	570	8	–	–
radius_pss_5	d	(0.0, 20.0)	1	578	8	–	–
good_pss_5	c	–	1	586	1	–	–
x_raw_pss_5	d	–	1	587	8	–	–
y_raw_pss_5	d	–	1	595	8	–	–
x_corr_pss_5	d	–	1	603	8	–	–
y_corr_pss_5	d	–	1	611	8	–	–
az_pss_5	d	(0.0, 360.0)	1	619	8	Angle	deg
el_pss_5	d	(0.0, 360.0)	1	627	8	Angle	deg
valid_pss_5	c	–	1	635	1	–	–
boxcar_size_adc_0	u	(1.0, 1000.0)	1	636	2	–	–
boxcar_size_adc_1	u	(1.0, 1000.0)	1	638	2	–	–
boxcar_size_adc_2	u	(1.0, 1000.0)	1	640	2	–	–
boxcar_size_adc_3	u	(1.0, 1000.0)	1	642	2	–	–
boxcar_size_adc_4	u	(1.0, 1000.0)	1	644	2	–	–
boxcar_size_adc_5	u	(1.0, 1000.0)	1	646	2	–	–

current_az	d	(-360.0, 360.0)	1	648	8	–	–
offset_az	d	(-360.0, 360.0)	1	656	8	–	–
best_az	d	(0.0, 360.0)	1	664	8	Angle	deg
best_el	d	(0.0, 360.0)	1	672	8	Angle	deg
timestamp_gps	d	–	1	680	8	Time	s
lat_gps	d	–	1	688	8	–	–
lon_gps	d	–	1	694	8	–	–
alt_gps	d	–	1	704	8	–	–
baud_gps	i	(0.0, 115200.0)	1	712	4	–	–
ra_sun_gps	d	–	1	716	8	–	–
dec_sun_gps	d	–	1	724	8	–	–
az_sun_gps	d	–	1	732	8	–	–
el_sun_gps	d	–	1	740	8	–	–
ha_sun_gps	d	–	1	748	8	–	–
array_tip	d	–	1	756	8	–	–
array_tilt	d	–	1	764	8	–	–
pos_table	d	–	1	772	8	Angle	deg
last_time_table	d	–	1	780	8	Time	s
at_target_table	c	–	1	788	1	–	–
cmd_position_table	d	(-360.0, 360.0)	1	789	8	Angle	deg
cmd_speed_table	d	–	(0.0, 90.0)	797	8	Speed	deg/s
set_pos_table	d	(-360.0, 360.0)	1	805	8	Angle	deg
cycle_start_table	d	(-360.0, 360.0)	1	813	8	Angle	deg
cycle_end_table	d	(-360.0, 360.0)	1	821	8	Angle	deg
cycles_left_table	i	–	1	829	4	–	cycles

Table 30: Telemetry Items – total frame size of 833 Bytes

8 Sample Programs & API

8.1 Connecting for the First Time

As mentioned in Section 7.2, Telemetry is transmitted continuously over port 54345 via UDP to the FDC. This port is configurable after the first connection, but must be used for initial setup purposes.

Below, a sample of a simple connection program in **Rust** is outlined:
Send address (default for commands): Address of the RPi: 10.0.0.2:54345
Receive address (default for telemetry): IP of the laptop: 10.0.0.1:53335

```
1 //When calling the function:
2 let port = 53335;
3 let socket = commands::connect(port.to_string()).unwrap();
4
5 //Command to establish the connection
6 pub fn connect(port: String) -> Result<UdpSocket, Box<dyn Error>> {
7     //creating socket
8     let socket = UdpSocket::bind(format!("0.0.0.0:{}", port))
9         .expect("Couldn't bind to address");
10
11     //return the socket
12     Ok(socket)
13 }
```

Another sample in **C** is below:

```
1 int connect_socket() {
2     int sockfd;
3     struct sockaddr_in server_addr;
4
5     // Creating socket
6     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
7     if (sockfd < 0) {
8         perror("Couldn't create socket");
9         exit(EXIT_FAILURE);
10    }
11
12    // Binding the socket to an address
13    memset(&server_addr, 0, sizeof(server_addr));
14    server_addr.sin_family = AF_INET;
15    server_addr.sin_addr.s_addr = INADDR_ANY;
16    server_addr.sin_port = htons(LOCAL_PORT);
17
18    if (bind(sockfd, (const struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
19        perror("Couldn't bind to address");
20        exit(EXIT_FAILURE);
21    }
22
23    return sockfd;
24 }
```

8.2 Parsing the Byte Stream

The byte stream is transmitted via the little endian system. This means it stores the least significant byte at the smallest address. Most data items are type f64 and 8 bytes, so this is important to consider for parsing the telemetry packets correctly.

A sample in **Rust** is outlined below:

```
1 //In Main -----
2 fn main() {
3     //send address
4     let addr_send = "10.0.0.2";
5
6     //recieve address
7     let addr_rec = "0.0.0.0:54345";
8 }
```

```

9 //connect
10 let port = "53335";
11
12 println!("Connecting to the device");
13 let socket = commands::connect(port.to_string()).unwrap();
14
15 println!("after connect");
16 //get a buffer to receive the datagram --> 772 bytes via the manual
17 let mut buf = [0; 772];
18
19 //reading continuously and exporting to .csv
20 loop{
21     //receive the data
22     match socket.recv_from(&mut buf) {
23         //match the result
24         Ok((received, _)) => {
25             //check if the data is the correct size
26             if(received != 772){
27                 //error handling
28                 eprintln!("Insufficient data received");
29             }else{
30                 println!("Data received");
31                 //read the data
32                 commands::read(&mut buf);
33             }
34         }
35         //error handling
36         Err(e) => eprintln!("Error: {:?}", e),
37     }
38 }
39 }
40 }
41
42 //Interpreting the byte stream-----
43 pub fn read(arr: &mut [u8]){
44     //process the array into the telemetry values based on the size array
45     let mut vec: Vec<f64> = Vec::new();
46     let mut arr_vec = arr.to_vec();
47     let mut used = Vec::new();
48     //for loop for each pss
49     for x in 0..6 {
50         //for loop for each telemetry value
51         // println!("x {}",x);
52         for i in 0..13{
53             //get sizes value
54             let size = SIZES[i];
55
56             //process that amount of bytes, reconstruct back into an integer
57             let mut val: f64 = 0.0;
58
59             if(size == 8){
60                 //get the 8 bytes
61                 val = f64::from_le_bytes([arr_vec[0], arr_vec[1], arr_vec[2],
62                 arr_vec[3], arr_vec[4], arr_vec[5], arr_vec[6], arr_vec[7]]);

```

```

63         //remove the used bytes
64         let temp = arr_vec.split_off(8);
65
66         used.push(arr_vec);
67
68         arr_vec = temp;
69
70     }else if size == 1{
71         //get the 1 byte
72         val = arr_vec[0] as f64;
73
74         //remove the used bytes
75         let temp = arr_vec.split_off(1);
76
77         used.push(arr_vec);
78
79         arr_vec = temp;
80
81     }
82
83     //append to the vector
84     vec.push(val);
85
86 }
87
88 }
89
90 //now process the remaining data, independant of each PSS
91 //boxcars --> 2 bytes each
92 for i in 0..6{
93     //process that amount of bytes, reconstruct back into an integer
94     let val;
95
96     //process and reconstruct
97     val = u16::from_le_bytes([arr_vec[0], arr_vec[1]]) as f64;
98
99     //push used values
100    let temp = arr_vec.split_off(2);
101
102    used.push(arr_vec);
103
104    arr_vec = temp;
105
106    //append to the vector
107    vec.push(val);
108 }
109
110 //print size of arr vec
111 // println!("size of arr_vec {}", arr_vec.len());
112
113 //remaining values are 8 bytes except baud gps
114 for i in 0..16{
115     //if i = 7, baud gps is 4 bytes to process
116     let val;

```

```

117
118     if i == 8{
119         //process 4 bytes
120         val = f32::from_le_bytes([arr_vec[0], arr_vec[1],
121             arr_vec[2], arr_vec[3]]) as f64;
122
123         //push used values
124         let temp = arr_vec.split_off(4);
125
126         used.push(arr_vec);
127
128         arr_vec = temp;
129
130         //append to the vector
131         vec.push(val);
132     }else{
133         //process 8 bytes
134         val = f64::from_le_bytes([arr_vec[0], arr_vec[1], arr_vec[2],
135             arr_vec[3], arr_vec[4], arr_vec[5], arr_vec[6], arr_vec[7]]);
136
137         //push used values
138         let temp = arr_vec.split_off(8);
139         used.push(arr_vec);
140
141         arr_vec = temp;
142
143         //append to the vector
144         vec.push(val);
145     }
146 }
147
148 //write each line to csv
149 append_csv(vec, "telemetry.csv");
150
151 }

```

Another sample in **C** is below:

```

1 // Constant size array for each set of PSS telemetry
2 int SIZES[13] = {8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 1};
3
4 // Function to read and interpret the byte stream
5 void read_telemetry(unsigned char *arr, int arr_len) {
6     double vec[35] = {0}; // Adjust size according to total expected telemetry values
7     int offset = 0;
8     int idx = 0;
9
10    // Process PSS telemetry
11    for (int x = 0; x < 6; ++x) {
12        for (int i = 0; i < 13; ++i) {
13            int size = SIZES[i];
14            double val = 0.0;
15
16            if (size == 8) {
17                memcpy(&val, arr + offset, sizeof(double));
18                offset += 8;
19            } else if (size == 1) {
20                val = arr[offset];
21                offset += 1;
22            }

```



```

23         vec[idx++] = val;
24     }
25 }
26
27
28 // Process boxcar telemetry
29 for (int i = 0; i < 6; ++i) {
30     uint16_t val;
31     memcpy(&val, arr + offset, sizeof(uint16_t));
32     vec[idx++] = val;
33     offset += 2;
34 }
35
36 // Process remaining data
37 for (int i = 0; i < 16; ++i) {
38     double val = 0.0;
39
40     if (i == 8) { // GPS baud rate
41         float val_float;
42         memcpy(&val_float, arr + offset, sizeof(float));
43         val = val_float;
44         offset += 4;
45     } else {
46         memcpy(&val, arr + offset, sizeof(double));
47         offset += 8;
48     }
49
50     vec[idx++] = val;
51 }
52
53 // Append to CSV
54 append_csv(vec, idx, "telemetry.csv");
55 }

```

8.3 Sending Commands

Sending commands is relatively simple, as they are ASCII based. If there are input parameters, they are to be space-separated, as follows:

adc_0_filter 20

String or character parameters are also space-separated, but must be quoted, as follows:

gps_connect 9600

Sample functions that send these commands are outlined below in **Rust**:

```

1 //set offset az
2 pub fn set_offset_az(az:f32, socket: &UdpSocket, addr: &str, port: &str){
3     let message = format!("set_offset_az_{}", az);
4     let m = message.as_bytes();
5
6     socket.send_to(&m,format!("{}",addr,port)).expect("Couldn't send data");
7 }
8
9 //connect gps
10 pub fn connect_gps(baud: i32, dev: &str, socket: &UdpSocket, addr: &str, port: &str){
11     let message = format!("gps_connect_{}_'", baud, dev);
12     let m = message.as_bytes();
13
14     socket.send_to(&m,format!("{}",addr,port)).expect("Couldn't send data");

```

```

15 }
16
17 //disconnect gps
18 pub fn disconnect_gps(socket: &UdpSocket, addr: &str, port: &str){
19     let message = format!("gps_disconnect");
20     let m = message.as_bytes();
21
22     socket.send_to(&m,format!("{}",addr,port)).expect("Couldn't send data");
23 }
24
25 //force reset of adc X
26 pub fn force_reset_adc(x:i16, socket: &UdpSocket, addr: &str, port: &str){
27     let message = format!("adc_{}_force_reset", x);
28     let m = message.as_bytes();
29
30     socket.send_to(&m,format!("{}",addr,port)).expect("Couldn't send data");
31 }

```

Another sample in C is below:

```

1 #code here
2 // Function to connect GPS
3 void connect_gps(int baud, const char *dev, int sockfd) {
4     char message[256];
5     struct sockaddr_in server_addr;
6
7     sprintf(message, "gps_connect %d '%s'", baud, dev);
8
9     memset(&server_addr, 0, sizeof(server_addr));
10    server_addr.sin_family = AF_INET;
11    server_addr.sin_port = htons(PORT);
12    server_addr.sin_addr.s_addr = inet_addr("10.0.0.2");
13
14    sendto(sockfd, message, strlen(message), MSG_CONFIRM, (const struct sockaddr *)&server_addr,
15           sizeof(server_addr));
16 }
17 // Function to disconnect GPS
18 void disconnect_gps(int sockfd) {
19     char message[256] = "gps_disconnect";
20     struct sockaddr_in server_addr;
21
22     memset(&server_addr, 0, sizeof(server_addr));
23     server_addr.sin_family = AF_INET;
24     server_addr.sin_port = htons(PORT);
25     server_addr.sin_addr.s_addr = inet_addr("10.0.0.2");
26
27     sendto(sockfd, message, strlen(message), MSG_CONFIRM, (const struct sockaddr *)&server_addr,
28           sizeof(server_addr));
29 }
30 // Function to force reset of ADC
31 void force_reset_adc(int x, int sockfd) {
32     char message[256];
33     struct sockaddr_in server_addr;
34
35     sprintf(message, "adc_%d_force_reset", x);
36
37     memset(&server_addr, 0, sizeof(server_addr));
38     server_addr.sin_family = AF_INET;
39     server_addr.sin_port = htons(PORT);
40     server_addr.sin_addr.s_addr = inet_addr("10.0.0.2");
41
42     sendto(sockfd, message, strlen(message), MSG_CONFIRM, (const struct sockaddr *)&server_addr,
43           sizeof(server_addr));

```

9 Calibration

9.1 Stage 1 Calibration - Individual PSS Linearization

In the first stage of calibration, the accuracy of each individual PSD sensor was evaluated at the sub micron level. To achieve this, a laser is pointed at the PSD to imitate a point source of light which can be expected from a pinhole. Then, the entire sensor surface area was mapped, and was then linearized based on position and generated heat maps (Figure 9). Thus, from this scanning, each sensor surface was able to be calibrated using the custom FSW to adjust for any inherent discontinuities or nonuniformities present. This allows for each PSS' sensing surface response to be symmetrical and consistent to any sun position.

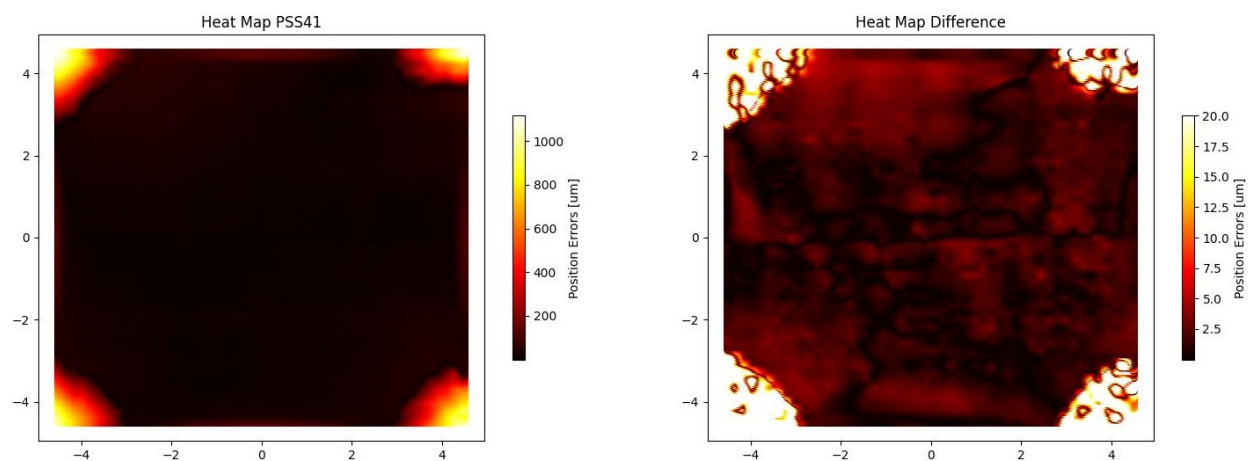


Figure 9: PSS 41 Heat Map

9.2 Stage 2 Calibration - Az/EI Correlation

In the second stage of calibration, the position sensor assembly is evaluated, specifically with respect to each other. This is an on-sky calibration and requires a cloud free sky to conduct the process. The sensors are now fully integrated into the housing and are rotated as the sun is within the elevation range, allowing the mapping of relative position to each other with a known sun position. As before, each sensor can be calibrated for discontinuities or nonuniformities, ensuring a sound overall scanning surface.

Through stage 1 and 2 calibration the PSS achieves a 5 arcminute resolution and accuracy.